Escape Room Platform Testing Plan

Testing Plan – Escape Room Platform

Status: Final

Author: Iyanuoluwa Olanipekun (QA / Tester)

Project: Escape Room Platform

Backend Framework: Django 5.2

Testing Frameworks: Pytest, pytest-django, Django REST Framework APIClient

1. Purpose of This Document

This Testing Plan documents the validation and verification strategy used for the Escape Room Platform. The objective of testing is to ensure that all functional requirements defined in the Product Requirements Document (PRD) are implemented correctly and that the backend system behaves reliably under normal, edge-case, and failure conditions.

This document describes the testing scope, tools, execution results, and traceability between requirements and tests.

2. ValidationStrategy (Are We Building the Right Product?)

2.1 Validation Approach

Validation focuses on ensuring that the system meets user needs and satisfies the functional requirements defined in the PRD and One-Pager.

The validation process included:

- Mapping functional requirements to test cases
- Designing tests around realistic gameplay scenarios
- Verifying observable behaviors such as puzzle progression, hint delivery, and session completion

Validation was achieved primarily through automated tests that simulate real gameplay interactions.

2.2 Acceptance Criteria

The following user-level acceptance criteria were validated:

- A player can start an escape room session
- A player can submit answers and progress through puzzles
- Hints escalate based on the number of failed attempts
- Correct answers advance or complete the session
- Gameplay statistics (attempts, completion state, time) are tracked accurately

3. Verification Strategy (Are We Building the Product Right?)

3.1 Verification Approach

Verification ensures that the backend implementation behaves correctly according to the PRD and technical design.

A layered testing strategy was used:

- **Unit Testing** – verification of isolated gameplay logic
- **Integration (API) Testing** – verification of HTTP endpoints and database interactions
- **Regression Testing** – re-execution of the full test suite after changes

4. Test Environment

	Component	Configuration
1	Operating System	Windows
2	Python Version	3.13

3	Django Version	5.2.13
4	Database	Django test database (SQLite)
5	Test Runner	Pytest 9.0.3
6	Plugins	pytest-django 4.12.0

5. Test Execution Summary

5.1 Unit Tests

Unit tests were executed using:

```shell
python -m pytest -v

```

Results:

- All unit tests passed successfully
- Core gameplay logic behaves as expected

6. Unit Test Coverage Overview

Unit tests validate the following backend logic:

- Answer checking (string, numeric, regex)
- Hint escalation rules
- Puzzle sequencing and branching logic
- Session progress and completion checks
- Timeout and failure handling

These tests primarily target functions implemented in `gameplay/game_logic.py`.

7. Testing Findings and Design Clarifications

During testing, minor ambiguities in the PRD were identified and clarified:

1. Branching puzzle answers are treated as case-insensitive for usability.
2. When a session times out, progress is preserved while the session is marked as failed.

These behaviors were confirmed as acceptable design decisions and documented through tests.

8. Regression Testing

Regression testing was conducted by re-running the full test suite after integration testing and configuration fixes.

Result: No regressions were detected.

9. Requirements Traceability Matrix (RTM)

The table below maps PRD functional requirements to **unit tests** and **backend integration (API) tests**, demonstrating traceability from requirements to verified implementation.

Integration Test Execution Status: A total of **5 backend API integration tests were executed with the following result: Status = PASSED (all tests passed successfully).**

	FR ID	Functional Requirement	Priority	Test Level	Test Case ID / Name	Status
1	FR2.1	Start a game session	High	Integration (API)	`test_st`	Passed

2	FR2.2	Submit puzzle answers	High	Unit, Integration (API)	`test_su`	Passed
3	FR2.3	Provide escalating hints	Medium	Unit, Integration (API)	`test_ge`	Passed
4	FR2.4	Track puzzle attempts	Medium	Unit	`test_up`	Passed
5	FR2.5	Detect session completion	High	Unit, Integration (API)	`test_ch`	Passed

Priority Definitions

- **High:** Core gameplay functionality; failure blocks user progress
- **Medium:** Important usability functionality; failure degrades experience

10. Summary of Test Results

- Unit tests: **Passed**
- Integration (API) tests: **Passed**
- Overall backend behavior verified against functional requirements

11. Integration (API) Testing Details

11.1 Scope

Backend integration testing was performed for authenticated REST API

endpoints exposed by the `gameplay` application. These tests validate routing, authentication enforcement, JSON request/response handling, and database persistence.

11.2 Tools and Approach

- **Framework:** Pytest with pytest-django
- **Client:** Django REST Framework APIClient
- **Authentication:** `APIClient.force_authenticate(user=...)` This approach enables realistic API-level validation without requiring live JWT issuance.

11.3 Endpoints Covered

- **`GET /api/start-session/POST /api/submit/GET /api/hint/`**

11.4 Assertions and Coverage

Integration tests assert:

- Correct HTTP status codes (200, 400, 401/403)
- Proper JSON response structure
- Authentication enforcement
- Correct database side effects

11.5 Relationship to Unit Tests

Integration tests ensure that gameplay logic validated at the unit level behaves correctly when accessed through external HTTP endpoints.

11.6 Limitations and Future Work

Frontend end-to-end (UI) testing was not performed due to incomplete frontend integration. UI testing is planned as future work once frontend components stabilize.